



Security Assessment Report

Orca Whirlpools

PR 1189 and 99

January 14, 2026

Summary

The Sec3 team was engaged to conduct a thorough security analysis of the PRs of the Orca Whirlpools program.

The artifact of the audit was the source code of the following programs, excluding tests, in <https://github.com/orca-so/whirlpools> and a private repository.

The initial audit focused on the following versions and revealed 2 issues or questions.

#	Task	Type	Commit
P1	PR1189: AMM-130 Add open one-sided position logic	Solana	aaf7aed (12/23/2025) → f0f4c53 (12/24/2025)
P2	PR99: AMM-174 update metadata url for dynamic position nfts	Solana	dc8ac8f (12/31/2025) → c06bb9f (01/06/2026)

This report provides a detailed description of the findings and their respective resolutions.

Table of Contents

Result Overview 3

Findings in Detail 4

 [P1-I-01] Duplicated tick checks in resolve_one_sided_position_ticks 4

 [P1-I-02] Full-range pool check improvements 6

Appendix: Methodology and Scope of Work 7

Result Overview

Issue	Impact	Status
PR1189: AMM-130 ADD OPEN ONE-SIDED POSITION LOGIC		
[P1-I-01] Duplicated tick checks in resolve_one_sided_position_ticks	Info	Resolved
[P1-I-02] Full-range pool check improvements	Info	Resolved
PR99: AMM-174 UPDATE METADATA URL FOR DYNAMIC POSITION NFTS		
No issues found		

Findings in Detail

PR1189: AMM-130 ADD OPEN ONE-SIDED POSITION LOGIC

[P1-I-01] Duplicated tick checks in resolve_one_sided_position_ticks

Identified in commit [f0f4c53](#).

In the `resolve_one_sided_position_ticks` function, the program derives the nearest usable tick indices above or below `current_sqrt_price` to serve as the LP position's lower or upper tick indices.

```
/* programs/whirlpool/src/util/shared.rs */
124 | if lower_is_sentinel {
125 |     // Snap just above the current sqrt price to stay single-sided.
126 |     let anchor_tick = if price_is_on_tick {
127 |         price_tick
128 |     } else {
129 |         price_tick + 1
130 |     };
131 |     let snapped = snap_tick_up(anchor_tick);
135 |     resolved_tick_lower_index = snapped;
136 | }
137 | if upper_is_sentinel {
138 |     // Snap just below the current sqrt price to stay single-sided.
139 |     let snapped = snap_tick_down(price_tick);
143 |     resolved_tick_upper_index = snapped;
144 | }
145 | if resolved_tick_lower_index >= resolved_tick_upper_index {
146 |     return Err(ErrorCode::InvalidTickIndex.into());
147 | }

/* programs/whirlpool/src/state/position.rs */
123 | fn validate_tick_range_for_whirlpool(
124 |     whirlpool: &Account<Whirlpool>,
125 |     tick_lower_index: i32,
126 |     tick_upper_index: i32,
127 | ) -> Result<()> {
128 |     if !Tick::check_is_usable_tick(tick_lower_index, whirlpool.tick_spacing)
129 |     || !Tick::check_is_usable_tick(tick_upper_index, whirlpool.tick_spacing)
130 |     || tick_lower_index >= tick_upper_index
131 |     {
132 |         return Err(ErrorCode::InvalidTickIndex.into());
133 |     }
}
```

The `resolved_tick_lower_index >= resolved_tick_upper_index` check duplicates the `validate_tick_range_for_whirlpool` validation performed during the `open_position` process, so that this check can be safely removed.

Resolution

Fixed by commit [5863e81](#).

PR1189: AMM-130 ADD OPEN ONE-SIDED POSITION LOGIC [P1-I-02] Full-range pool check improvements

Identified in commit [f0f4c53](#).

The `resolve_one_sided_position_ticks` function derives the lower and upper tick indices for a single-sided liquidity position. It's not supposed to be applied to full-range pools.

In the current implementation, the actual enforcement for full-range pools happens inside `validate_tick_range_for_whirlpool` (called by `position.open_position`), after the `resolve_one_sided_position_ticks`.

```
/* programs/whirlpool/src/instructions/open_position_with_metadata.rs */
093 | let (resolved_tick_lower_index, resolved_tick_upper_index) = resolve_one_sided_position_ticks(
094 |     tick_lower_index,
095 |     tick_upper_index,
096 |     whirlpool.tick_spacing,
097 |     whirlpool.sqrt_price,
098 | )?;
099 |
100 | position.open_position(
101 |     whirlpool,
102 |     position_mint.key(),
103 |     resolved_tick_lower_index,
104 |     resolved_tick_upper_index,
105 | )?;
```

However, applying `resolve_one_sided_position_ticks` to full-range pools is not a correctness issue. At worst, the resolved indices become non-full-range bounds that then fail the full-range check in `position.open_position`. So the full-range requirement cannot be bypassed.

It could be a UX/consistency improvement to add a guard in `resolve_one_sided_position_ticks` to return a clearer error for full-range pools.

Resolution

Fixed by commit [a5a7319](#).

Appendix: Methodology and Scope of Work

Assisted by the Sec3 Scanner developed in-house, the manual audit particularly focused on the following work items:

- Check common security issues.
- Check program logic implementation against available design specifications.
- Check poor coding practices and unsafe behavior.
- The soundness of the economics design and algorithm is out of scope of this work

DISCLAIMER

The instance report ("Report") was prepared pursuant to an agreement between Coderect Inc. d/b/a Sec3 (the "Company") and Orca Management Company, S.A (the "Client"). This Report solely includes the results of a technical assessment of a specific build and/or version of the Client's code specified in the Report ("Assessed Code") by the Company. The sole purpose of the Report is to provide the Client with the results of the technical assessment of the Assessed Code. The Report does not apply to any other version and/or build of the Assessed Code. Regardless of the contents of the Report, the Report does not (and should not be interpreted to) provide any warranty, representation or covenant that the Assessed Code: (i) is error and/or bug free, (ii) has no security vulnerabilities, and/or (iii) does not infringe any third-party rights. Moreover, the Report is not, and should not be considered, an endorsement by the Company of the Assessed Code and/or of the Client. Finally, the Report should not be considered investment advice or a recommendation to invest in the Assessed Code and/or the Client.

This Report is considered null and void if the Report (or any portion thereof) is altered in any manner.

ABOUT

The Sec3 audit team comprises a group of computer science professors, researchers, and industry veterans with extensive experience in smart contract security, program analysis, testing, and formal verification. We are also building automated security tools that incorporate static analysis, penetration testing, and formal verification.

At Sec3, we identify and eliminate security vulnerabilities through the most rigorous process and aided by the most advanced analysis tools.

For more information, check out our [website](#) and follow us on [twitter](#).

